

Ingeniería del Software II

Práctica #8 – Análisis de Punteros

Ejercicio 1

Dado el siguiente programa:

```
1 A a = new A(); // call site #1
2 B b = new B(); // call site #2
3 a.f = b;
4 c = a;
5 c.f = a;
6 c = b;
```

- Calcular el CFG.
- Calcular el PTG al final del programa utilizando un análisis points-to flow-sensitive.
- Calcular el PTG utilizando el algoritmo de Andersen.
- Ejecutar de nuevo los puntos 2 y 3 sobre el mismo programa pero intercambiando las líneas 4 y 6. ¿Qué diferencias se ven en cada caso? ¿Qué afirmación confirma este pequeño experimento?
- Calcular el PTG utilizando el algoritmo de Steensgard.

Ejercicio 2

Dado el siguiente programa (suponiendo que la subclase B es de A):

```
1 void main() {
2   A a = new A();
3   B b = new B();
4   A c = m1(a);
5   A d = m2(b);
6 }
7 A m1(A p) {
8   A e = m2(p);
9   return e;
10 }
11 A m2(A k) {
12   B h = new B();
13   h.f = k;
14   return h;
15 }
```

Calcular el PTG luego de ejecutar la línea 5 del método `main`:

- Sin usar contexto para las alocações en memoria.
- Usando cadenas de llamada de tamaño = 1
- Usando cadenas de llamada de tamaño = 2

Ejercicio 3

Dado el siguiente programa (asumiendo que H1 y H2 son subclases de H)

```

1  void main() {
2      x = new H1;
3      z = new H2;
4      y = f(x);
5      w = f(z);
6  }
7  H f(H v) {
8      u = v;
9      return u;
10 }

```

- Calcular el PTG para el final del método `main` usando el algoritmo de Andersen sin contexto.
- Calcular el PTG luego de hacer clonig del método `f`.
- Calcular el PTG usando como contextos cadenas de llamada de tamaño = 1.

Ejercicio 4

Dado el siguiente programa (asumiendo que H1 y H2 son subclases de H)

```

1  void main() {
2      x = new H1();
3      z = new H2();
4      y = f(x);
5      w = f(z);
6  }
7  H f(H v) {
8      if (v == null) {
9          v = f(v)
10     }
11     return v;
12 }

```

- Calcular el PTG para el final del método `main` usando el algoritmo de Andersen sin contexto.
- ¿Se puede hacer clonig?
- Calcular el PTG usando como contexto cadenas de llamada de longitud $k=2$.
- Calcular el PTG usando contexto funcional (observación: el contexto funcional en este caso es un points-to graph).

Ejercicio 5

Dado el siguiente programa Java:

```

1  class Elem { Elem siguiente; }
2
3  void main() {
4      Elem e1 = new Elem(); // E1
5      Elem e2 = new Elem(); // E2
6      Elem e3 = new Elem(); // E3
7      e1.siguiente = e2;
8      e2.siguiente = e3;
9      e3.siguiente = e1;
10     e3 = e3.siguiente;
11 }

```

- a. Generar restricciones de Andersen para el programa
- b. Resolver el sistema paso a paso
- c. Dibujar el points-to graph resultante

Ejercicio 6

Dado el siguiente programa en Java:

```
1      class A { A f; };
2
3      int m(int i) {
4          A a = new A(); // A1
5          A b = new A(); // A2
6          A c = new A(); // A3
7          A d = a;
8          if(i>0) {
9              d = b;
10         }
11         d.f = c;
12         A e = a.f;
13     }
```

- a. Calcular las restricciones que genera el análisis de Steensgard para este programa.
- b. Encontrar la solución más exacta al sistema de restricciones.
- c. Dibujar el points-to graph resultante (sin unificación).